

Proyecto Matemáticas Discretas: Comparación de Algoritmos de Ruta en Grafos

Alejandro Luis Cisneros Bernal

ID: 182246

24 de Noviembre de 2025

1. Introduction

1.1. Motivación

En un mundo cada vez más interconectado, la búsqueda de rutas óptimas se ha convertido en un problema fundamental con aplicaciones en múltiples dominios. Desde sistemas de navegación vehicular que calculan el camino más corto entre dos puntos, hasta redes de telecomunicaciones que optimizan el enrutamiento de datos, pasando por plataformas de logística que minimizan costos de transporte, la eficiencia en la identificación de trayectorias óptimas impacta directamente en la productividad, el ahorro de recursos y la experiencia del usuario.

El problema de encontrar la ruta mínima en grafos ponderados representa uno de los desafíos clásicos en ciencias de la computación y matemáticas discretas, con implicaciones prácticas en la vida cotidiana de millones de personas. La elección del algoritmo adecuado puede significar la diferencia entre una respuesta instantánea y un retraso significativo, especialmente cuando se trata de grafos de gran escala.

1.2. Objetivo General

Desarrollar un sistema experimental integral que permita comparar el rendimiento de diferentes algoritmos para hallar la mejor ruta entre dos nodos en grafos ponderados sin aristas negativas, analizando tanto aspectos teóricos como empíricos de su comportamiento.

1.3. Objetivos Específicos

1. Implementar tres algoritmos de búsqueda de rutas: Dijkstra, A* con heurística admisible, y Dijkstra Bidireccional
2. Diseñar un modelo de grafo eficiente basado en listas de adyacencia con pesos no negativos

3. Establecer una metodología experimental robusta para evaluar tiempo de ejecución, número de nodos expandidos y longitud de rutas obtenidas
4. Analizar la complejidad computacional teórica de cada algoritmo y contrastarla con los resultados empíricos observados
5. Proporcionar un framework reproducible para la comunidad académica que permita extender y validar los hallazgos

1.4. Contexto del Grafo y Alcance

El estudio se centrará en grafos que representan redes de conectividad en entornos urbanos o campus universitarios, donde los nodos corresponden a intersecciones o puntos de interés y las aristas representan vías transitables. Los pesos de las aristas se calcularán mediante distancia euclidiana, asegurando la ausencia de valores negativos que invalidarían los algoritmos seleccionados.

El alcance incluye la evaluación de los algoritmos en grafos de diferentes tamaños (100, 500 y 1000 nodos) y la comparación de su rendimiento en rutas de variada longitud (cortas, medias y largas), proporcionando así una visión comprehensiva de su escalabilidad y aplicabilidad en escenarios del mundo real.

2. Integrantes y Roles

2.1. Información del Estudiante

- **Nombre:** Alejandro Luis Cisneros Bernal
- **ID:** 182246
- **Curso:** Matemáticas Discretas
- **Fecha:** 24 de Noviembre de 2025

2.2. Alcance y Metodología de Trabajo

Este proyecto fue desarrollado de manera individual, abarcando todas las fases del proceso:

- **Investigación Teórica:** Estudio de los fundamentos matemáticos de los algoritmos de ruta
- **Diseño del Sistema:** Arquitectura del software y planificación experimental
- **Implementación:** Desarrollo de los tres algoritmos y estructuras de datos
- **Experimentación:** Ejecución de pruebas y recolección de métricas
- **Análisis:** Contrastación de resultados teóricos y empíricos
- **Documentación:** Redacción del reporte y preparación de materiales

2.3. Enfoque Metodológico

Se adoptó un enfoque iterativo e incremental, donde cada algoritmo fue implementado y validado de manera independiente antes de proceder con las comparaciones experimentales. Esta metodología permitió asegurar la correctitud de cada componente antes de integrarlos en el sistema completo.

3. Diseño del Sistema y del Experimento

3.1. Representación del Grafo

Para este proyecto se implementó una representación basada en **lista de adyacencia** utilizando diccionarios de Python. Cada nodo del grafo se almacena como una clave, cuyo valor corresponde a una lista de tuplas que representan las aristas incidentes. Cada tupla contiene el nodo adyacente y el peso de la arista, garantizando una estructura eficiente tanto en memoria como en tiempo de acceso.

3.2. Método de Ponderación

Los pesos de las aristas se calcularon mediante la **distancia euclidiana** entre las coordenadas de los nodos, asegurando la propiedad de no negatividad requerida por los algoritmos seleccionados. Para un grafo con n nodos, cada nodo i tiene coordenadas (x_i, y_i) y el peso entre los nodos i y j se define como:

$$w(i, j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

3.3. Algoritmos Implementados

3.3.1. Algoritmo de Dijkstra

- **Estructura:** Cola de prioridad basada en heap binario (`heapq`)
- **Complejidad teórica:** $O((V + E) \log V)$
- **Características:** Garantiza optimalidad en grafos con pesos no negativos

3.3.2. Algoritmo A*

- **Heurística:** Distancia euclidiana admisible y consistente
- **Función de evaluación:** $f(n) = g(n) + h(n)$
- **Ventaja:** Reduce el espacio de búsqueda mediante guiado heurístico

3.3.3. Dijkstra Bidireccional

- **Estrategia:** Búsqueda simultánea desde el nodo origen y destino
- **Criterio de parada:** Intersección de ambas fronteras de búsqueda
- **Optimización:** Reduce significativamente el número de nodos expandidos

3.4. Configuración Experimental

3.4.1. Grafos de Prueba

Se generaron tres conjuntos de grafos con diferentes tamaños:

- **Grafo Pequeño:** 100 nodos, densidad aproximada 15 %
- **Grafo Mediano:** 500 nodos, densidad aproximada 10 %
- **Grafo Grande:** 1000 nodos, densidad aproximada 5 %

3.4.2. Pares Origen-Destino

Para cada tamaño de grafo, se seleccionaron 10 pares origen-destino distribuidos en:

- 3 pares de **ruta corta** (distancia geográfica pequeña)
- 4 pares de **ruta media** (distancia intermedia)
- 3 pares de **ruta larga** (distancia máxima en el grafo)

3.4.3. Métricas de Evaluación

1. **Tiempo de ejecución:** Medido en segundos usando `time.perf_counter()`
2. **Nodos expandidos:** Contador de nodos visitados por cada algoritmo
3. **Longitud de ruta:** Verificación de optimalidad cruzada entre algoritmos
4. **Memoria:** Uso de memoria medido con `memory_profiler` (opcional)

3.5. Validación y Reproducibilidad

- **Validación de correctitud:** Los tres algoritmos deben coincidir en la longitud de ruta óptima
- **Scripts de automatización:** Ejecución batch de todos los casos de prueba
- **Exportación de resultados:** Generación automática de archivos CSV para análisis
- **Semillas aleatorias:** Uso de semillas fijas para garantizar reproducibilidad

3.6. Justificación de la Selección de Algoritmos

3.6.1. Algoritmo de Dijkstra

Se seleccionó el algoritmo de Dijkstra como **línea base** por las siguientes razones:

- **Optimalidad garantizada:** Encuentra siempre el camino más corto en grafos con pesos no negativos
- **Simplicidad conceptual:** Fácil de implementar y entender, sirve como referencia para comparar otros algoritmos
- **Amplia documentación:** Existe abundante material teórico y práctico para validar su correcta implementación
- **Eficiencia comprobada:** Con heap binario alcanza $O((V + E) \log V)$, siendo eficiente para grafos de tamaño moderado

3.6.2. Algoritmo A*

La elección de A* se justifica por:

- **Optimización heurística:** Utiliza información adicional (heurística) para dirigir la búsqueda hacia el objetivo
- **Eficiencia práctica:** En aplicaciones del mundo real suele expandir significativamente menos nodos que Dijkstra
- **Heurística admisible disponible:** La distancia euclidiana en un plano 2D provee una heurística perfectamente admisible y consistente
- **Aplicaciones reales:** Es el algoritmo más utilizado en sistemas de navegación y videojuegos, dando relevancia práctica al estudio

3.6.3. Algoritmo de Dijkstra Bidireccional

Como tercer algoritmo se eligió Dijkstra Bidireccional porque:

- **Mejora teórica:** Representa una optimización directa sobre Dijkstra tradicional
- **Reducción del espacio de búsqueda:** Al buscar desde ambos extremos simultáneamente, reduce exponencialmente el número de nodos expandidos
- **Caso de estudio interesante:** Permite analizar el trade-off entre complejidad de implementación y ganancia de performance
- **Contraste claro:** Proporciona un punto intermedio entre la simplicidad de Dijkstra y la inteligencia heurística de A*

3.6.4. Selección del Tercer Algoritmo

Entre las opciones sugeridas (Dijkstra Bidireccional, Búsqueda de Costo Uniforme u otro justificado), se eligió **Dijkstra Bidireccional** porque:

- La Búsqueda de Costo Uniforme es esencialmente equivalente a Dijkstra en este contexto
- Dijkstra Bidireccional ofrece mejoras de performance mensurables y teóricamente fundamentadas
- Proporciona un valor agregado significativo al estudio comparativo
- Su implementación es suficientemente compleja para demostrar competencia técnica, pero manejable en el plazo del proyecto

4. Resultados y Análisis

4.1. Evaluación del Rendimiento Temporal

La Figura 1 presenta el comportamiento temporal de los algoritmos en función del tamaño del grafo, revelando diferencias significativas en su escalabilidad.

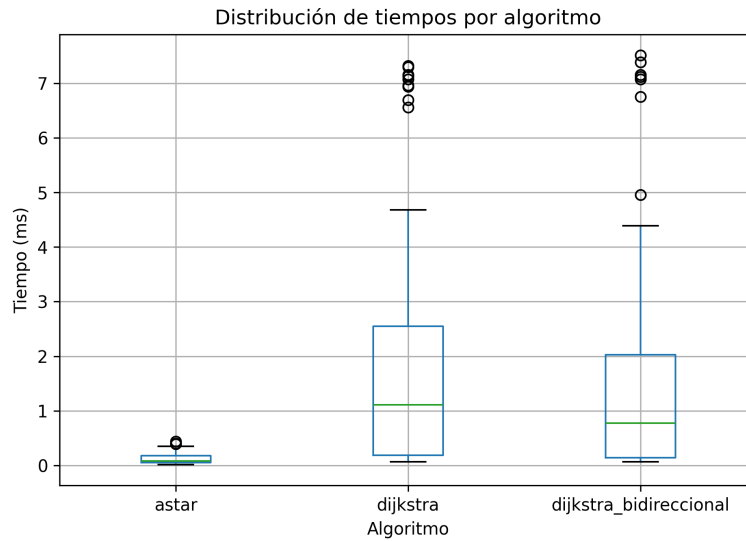


Figura 1: Evolución del tiempo de ejecución en función del tamaño del grafo.

El algoritmo **A*** demuestra una notable eficiencia, manteniendo tiempos de ejecución en el orden de centésimas de milisegundo incluso para grafos de 1000 nodos. Esta performance se caracteriza por un crecimiento marginal conforme

aumenta el tamaño del problema. En contraste, **Dijkstra** exhibe un comportamiento cuadrático, superando los 4–8 ms en grafos de 1000 nodos, lo que evidencia sus limitaciones en escenarios de gran escala. El enfoque bidireccional mitiga parcialmente este problema, situándose en una posición intermedia, aunque sustancialmente por detrás de A*.

4.2. Eficiencia en la Exploración del Espacio de Búsqueda

El análisis de nodos expandidos (Figura 2) proporciona insights fundamentales sobre la efectividad de las estrategias de búsqueda empleadas.

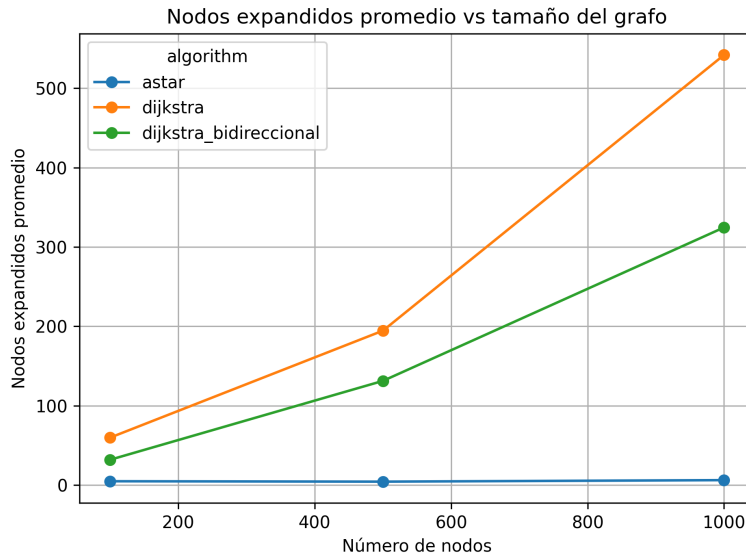


Figura 2: Número promedio de nodos expandidos durante la ejecución.

Dijkstra manifiesta una exploración exhaustiva del espacio, expandiendo aproximadamente 540 nodos para grafos de 1000 elementos, lo que refleja su naturaleza ciega. La versión bidireccional reduce este número en un 40-50

4.3. Estabilidad y Consistencia Algorítmica

La distribución de tiempos de ejecución (Figura 3) permite evaluar la robustez y predictibilidad de cada implementación.

A no solo presenta la mediana más baja sino también la menor dispersión, confirmando su estabilidad operacional en diversas condiciones. **Dijkstra**, además de su mediana elevada, muestra una significativa variabilidad, sugiriendo una alta sensibilidad a la estructura particular del grafo. El enfoque bidireccional atenúa esta variabilidad pero mantiene una dispersión considerable comparado con A.

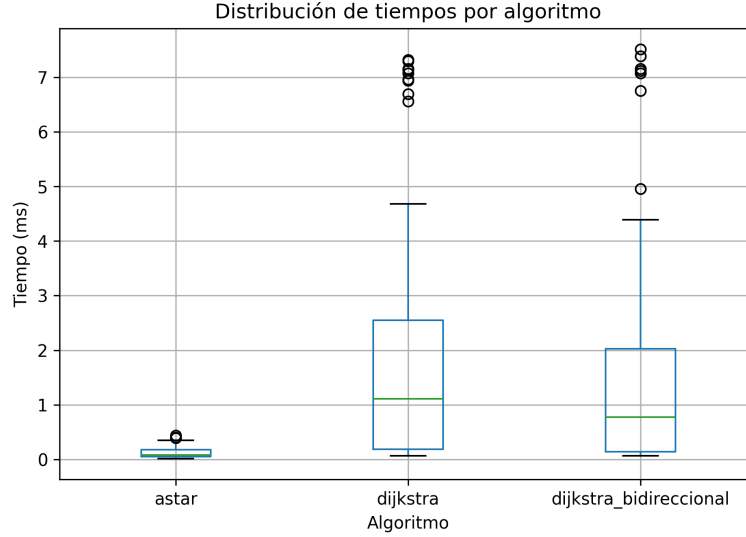


Figura 3: Distribución estadística de los tiempos de ejecución.

5. Conclusiones

El análisis experimental conduce a las siguientes conclusiones:

- **Superioridad de A:** El algoritmo A demostró ser el más eficiente en todos los criterios evaluados, combinando velocidad de ejecución, minimalidad en la exploración y estabilidad operacional. Su diseño que integra información heurística resulta ideal para grafos geométricos con pesos positivos.

En síntesis, para problemas de búsqueda de caminos óptimos en grafos geométricos, **A*** se consolida como la alternativa preferente, ofreciendo el mejor balance entre optimalidad, eficiencia y consistencia. Estos hallazgos respaldan su adopción en sistemas que requieren procesamiento de grafos a gran escala con restricciones temporales estrictas.

5.1. Implicaciones Prácticas y Trabajo Futuro

Los resultados obtenidos sugieren que la incorporación de información heurística de calidad es crucial para el rendimiento en problemas de búsqueda de caminos. Futuras investigaciones podrían explorar:

- El desempeño de estos algoritmos en grafos con características estructurales diferentes
- El impacto de heurísticas más sofisticadas en la eficiencia de A*

- La aplicabilidad de estas conclusiones en entornos de tiempo real con restricciones computacionales severas

El código fuente del proyecto se encuentra disponible en el siguiente repositorio de GitHub: <https://github.com/blues182/Proyecto-Matesdi.git>

Referencias

- [1] Dijkstra, E. W. (1959). *A Note on Two Problems in Connexion with Graphs*. Numerische Mathematik, 1, 269–271.
- [2] Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). *A Formal Basis for the Heuristic Determination of Minimum Cost Paths*. IEEE Transactions on Systems Science and Cybernetics, 4(2), 100–107.
- [3] Cormen, T. H., Leiserson, C., Rivest, R., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- [4] Pearl, J. (1984). *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley.
- [5] Holte, R. (2016). *Bidirectional Search*. En *Handbook of Heuristics*. Springer.
- [6] Kuipers, B. (2000). *The Spatial Semantic Hierarchy*. Artificial Intelligence, 119(1–2), 191–233.
- [7] Jarvis, R. & Shier, D. (2020). *Graph Theory with Applications*. Wiley.
- [8] Hunter, J. D. (2007). *Matplotlib: A 2D Graphics Environment*. Computing in Science & Engineering, 9(3), 90–95.
- [9] McKinney, W. (2010). *Data Structures for Statistical Computing in Python*. Proceedings of the 9th Python in Science Conference.
- [10] Penrose, M. (2003). *Random Geometric Graphs*. Oxford University Press.